

2nd Week

파이썬 기초

(내장 자료구조 & 함수 & 파일 & 운영체제)





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!



Jupyter Notebook 단축키

Keyboard shortcuts of Jupyter Notebook

Keyboard shortcuts

The Jupyter Notebook has two different keyboard input modes. **Edit mode** allows you to type code/text into a cell and is indicated by a green cell border. **Command mode** binds the keyboard to notebook level actions and is indicated by a grey cell border with a blue left margin.

Command Mode (press **Esc** to enable)

F: find and replace
Ctrl-Shift-P: open the command palette
Enter: enter edit mode
Shift-Enter: run cell, select below
Ctrl-Enter: run selected cells
Alt-Enter: run cell, insert below
Y: to code
M: to markdown
R: to raw
1: to heading 1
2: to heading 2
3: to heading 3
4: to heading 4
5: to heading 5
6: to heading 6
K: select cell above
Up: select cell above
Down: select cell below
J: select cell below

Edit Mode (press **Enter** to enable)

Tab: code completion or indent
Shift-Tab: tooltip
Ctrl-]: indent
Ctrl-[: dedent
Ctrl-A: select all
Ctrl-Z: undo
Ctrl-Shift-Z: redo
Ctrl-Y: redo
Ctrl-Home: go to cell start
Ctrl-Up: go to cell start
Ctrl-End: go to cell end
Ctrl-Down: go to cell end
Ctrl-Left: go one word left
Ctrl-Right: go one word right
Ctrl-Backspace: delete word before
Ctrl-Delete: delete word after
Ctrl-M: command mode
Ctrl-Shift-P: open the command palette
Esc: command mode

Shift-K: extend selected cells above
Shift-Up: extend selected cells above
Shift-Down: extend selected cells below
Shift-J: extend selected cells below
A: insert cell above
B: insert cell below
X: cut selected cells
C: copy selected cells
Shift-V: paste cells above
V: paste cells below
Z: undo cell deletion
D, **D**: delete selected cells
Shift-M: merge selected cells, or current cell with cell below if only one cell selected
Ctrl-S: Save and Checkpoint
S: Save and Checkpoint
L: toggle line numbers
O: toggle output of selected cells
Shift-O: toggle output scrolling of selected cells
H: show keyboard shortcuts
I, **I**: interrupt kernel
Q, **Q**: restart the kernel (with dialog)
Esc: close the pager
Q: close the pager
Shift-Space: scroll notebook up
Space: scroll notebook down

Shift-Enter: run cell, select below
Ctrl-Enter: run selected cells
Alt-Enter: run cell, insert below
Ctrl-Shift-Minus: split cell
Ctrl-S: Save and Checkpoint
Down: move cursor down
Up: move cursor up



Google Colab 단축키

Keyboard shortcuts of Google Colab

키보드 환경설정

단축키

단축키를 추가하거나 변경하려면 키 조합을 클릭한 다음 새 키를 입력하세요. Ctrl+M을(를) 다중 카-이벤트 단축키의 접두어로 사용할 수 있습니다.

단축키 설정	.ipynb 다운로드	단축키 설정	세션 다시 시작 및 모두 실행
단축키 설정	.py 다운로드	단축키 설정	색션 제목 셀 추가
단축키 설정	Colab Enterprise 열기	단축키 설정	셀 또는 선택 항목 복사
단축키 설정	Drive에 사본 저장	단축키 설정	셀 또는 선택 항목 잘라내기
단축키 설정	Drive에서 노트북 이동	Ctrl+Click	셀 선택 도구
단축키 설정	Enable auto-run of first cell or section on any execution	Ctrl+Shift+Y	셀 작업 다시 실행
단축키 설정	GitHub Gist로 사본 저장	Ctrl+M D	셀/선택항목 삭제
단축키 설정	GitHub에 사본 저장	단축키 설정	셀로 이동
단축키 설정	Google Drive의 노트 별표표시/별표표시 해제	Shift+Enter	셀을 실행하고 다음 셀 선택
단축키 설정	JSON 노트북 보기	Alt+Enter	셀을 실행하고 새 셀 삽입
단축키 설정	Show Colab AI	단축키 설정	스크래치 셀로 복사
단축키 설정	노트 설정 열기	Ctrl+Alt+N	스크래치 코드 셀 열기
단축키 설정	노트 업로드	단축키 설정	실습 모드에서 열기
Ctrl+O	노트 열기	Ctrl+M I	실행 중단
Ctrl+P	노트 인쇄	단축키 설정	실행된 코드 기록 표시
Ctrl+S	노트 저장	Ctrl+M B	아래에 코드 셀 삽입
단축키 설정	노트 출처 보기	Ctrl+M F	양식 보기 순환
단축키 설정	노트 파일 정보 표시	단축키 설정	양식 속성 수정
단축키 설정	노트를 휴지통으로 이동	단축키 설정	양식 입력란 추가
단축키 설정	노트북 공유	단축키 설정	양식 추가
Ctrl+F9	노트의 모든 셀 실행	단축키 설정	예약된 노트북 표시
단축키 설정	다시 연결	단축키 설정	왼쪽 창을 탭으로 이동
Ctrl+M N	다음 셀	Ctrl+M A	위에 코드 셀 삽입
Shift+Down	다음 셀을 포함하도록 선택 영역을 확장합니다.	단축키 설정	의견 사이드바
Ctrl+G	다음 찾기	Ctrl+M P	이전 셀
Ctrl+Shift+]	다음 탭으로 포커스 이동	Shift+Up	이전 셀을 포함하도록 선택 영역을 확장합니다.
단축키 설정	다음 탭으로 포커스 이동	Ctrl+Shift+G	이전 찾기
단축키 설정	다음 탭으로 포커스 이동	Ctrl+Shift+[	이전 탭으로 포커스 이동

Shift+Click	다중 셀 선택
단축키 설정	단일 탭 레이아웃 보기
Ctrl+M H	단축키 표시
단축키 설정	댓글 창 열기
Ctrl+Alt+M	댓글 추가
단축키 설정	드라이브 마운트
단축키 설정	드라이브 마운트 해제
단축키 설정	드라이브에서 찾기
단축키 설정	런타임 로그 보기
단축키 설정	런타임 연결 해제 및 삭제
단축키 설정	런타임에 연결
단축키 설정	로컬 런타임에 연결
단축키 설정	리소스 보기

Ctrl+M Z 또는 Ctrl+Shift+Z 마지막 셀 작업 실행취소

단축키 설정	마지막으로 실행한 셀로 포커스 이동
단축키 설정	맞춤 GCE VM에 연결
Ctrl+Shift+P	명령 팔레트 표시
단축키 설정	모든 런타임 초기화
Ctrl+]	모든 섹션/선택된 섹션 접기
Ctrl+[	모든 섹션/선택한 섹션 펼치기
Ctrl+Shift+A	모든 셀 선택
단축키 설정	모든 출력 지우기
단축키 설정	목차 표시
단축키 설정	버전 기록 표시
Ctrl+M S	버전 저장 및 고정
단축키 설정	변수 검사기 표시
단축키 설정	비교 노트북
단축키 설정	사용자 보안 비밀번호 열기
단축키 설정	상태 표시줄 표시/숨기기
단축키 설정	새 노트 만들기
단축키 설정	선택된 셀 병합

Ctrl+Space 또는 Tab 자동완성 (?)

Ctrl+H	전체 찾기/바꾸기
단축키 설정	접힌 색선 레이아웃 저장
Ctrl+Shift+S	초점이 맞춰진 셀 선택
Ctrl+Enter	초점이 맞춰진 셀 실행
단축키 설정	초점이 맞춰진 셀과 다음 셀 병합
단축키 설정	초점이 맞춰진 셀과 이전 셀 병합
단축키 설정	출력 전체화면 보기
Ctrl+M O	출력 표시/숨기기
Ctrl+M -	커서에서 분할

Tab	코드 문서화(docstring) 도움말을 토글
-----	------------------------------

⑦

단축키 설정	코드 생성
단축키 설정	코드 셀 추가
Ctrl+M Y	코드 셀로 변환
Ctrl+Alt+P	코드 스니펫 창 표시
단축키 설정	코드 표시/숨기기
단축키 설정	탭에서 셀 미러링
단축키 설정	탭으로 포커스 이동 여부 전환
단축키 설정	탭을 다음 창으로 이동
단축키 설정	탭을 열 2개로 보기
단축키 설정	탭을 이전 창으로 이동
단축키 설정	탭을 행 2개로 보기
단축키 설정	터미널 보기
단축키 설정	텍스트 셀 추가
Ctrl+M M	텍스트 셀로 변환
단축키 설정	파일 브라우저 표시
단축키 설정	편집기 설정 열기
Ctrl+M L	행 번호 전환
단축키 설정	헤더 공개 상태 전환
Ctrl+M	현재 셀에서 편집기/자기 저장

Ctrl+F10	선택된 셀을 실행하고 그 이후 모든 셀 실행	Ctrl	현재 선택된 셀에서 7바이트 단위로
		Esc	현재 셀 선택 해제
Ctrl+M K	선택된 셀을 위로 이동	Ctrl+F8	현재 셀 이전에 셀 실행
Ctrl+M J	선택된 셀을 아래로 이동	Shift+Ctrl+H	현재 셀에서 모두 바꾸기
단축키 설정	선택한 출력 지우기	Shift+Tab	현재 행 들여쓰기 취소
Ctrl+Shift+Enter	선택항목 실행	Ctrl/+	현재 행 주석 처리하기
단축키 설정	설정 열기	단축키 설정	호스팅된 런타임에 연결
단축키 설정	세션 관리		
Ctrl+M .	세션 다시 시작		

13

1부: 내장 자료구조

- 1) 자료구조
- 2) 순차 자료형





수업 때 다룰 내용

Contents in Lecture

이 장에서는 이 책 전반에 걸쳐 사용하게 될 파이썬 언어에 내장되어 있는 기능을 알아보자. pandas나 NumPy 같은 애드온 라이브러리는 대규모 데이터 계산을 위한 진보된 기능을 제공하지만, 내장되어 있는 기능은 파이썬 내장 자료 처리 도구와 함께 사용해야 한다.

먼저 파이썬의 기본 자료구조인 튜플, 리스트, 사전 그리고 집합부터 알아보고 재사용 가능한 파이썬 함수를 작성하는 방법을 살펴본다. 마지막으로 파이썬의 file 객체의 원리를 살펴보고 하드디스크에 직접 파일을 읽고 쓰는 방식을 알아보자.

파이썬의 자료구조는 단순하지만 강력하다. 자료구조의 사용법을 숙지하는 것이 파이썬의 고수가 되는 지름길이다.



자료구조와 순차 자료형

Data Structures and Sequences

Tuple

A tuple is a fixed-length, immutable sequence of Python objects. The easiest way to create one is with a comma-separated sequence of values:

```
In [1]: tup = 4, 5, 6
```

```
In [2]: tup  
Out[2]: (4, 5, 6)
```

When you're defining tuples in more complicated expressions, it's often necessary to enclose the values in parentheses, as in this example of creating a tuple of tuples:

```
In [3]: nested_tup = (4, 5, 6), (7, 8)
```

```
In [4]: nested_tup  
Out[4]: ((4, 5, 6), (7, 8))
```

You can convert any sequence or iterator to a tuple by invoking tuple:

```
In [5]: tuple([4, 0, 2])  
Out[5]: (4, 0, 2)
```

```
In [6]: tup = tuple('string')
```

```
In [7]: tup  
Out[7]: ('s', 't', 'r', 'i', 'n', 'g')
```

Elements can be accessed with square brackets [] as with most other sequence types. As in C, C++, Java, and many other languages, sequences are 0-indexed in Python:

```
In [8]: tup[0]  
Out[8]: 's'
```

While the objects stored in a tuple may be mutable themselves, once the tuple is created it's not possible to modify which object is stored in each slot:

```
In [9]: tup = tuple(['foo', [1, 2], True])
```

```
In [10]: tup[2] = False
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-10-c7308343b841> in <module>()  
----> 1 tup[2] = False  
TypeError: 'tuple' object does not support item assignment
```

If an object inside a tuple is mutable, such as a list, you can modify it in-place:

```
In [11]: tup[1].append(3)
```

```
In [12]: tup  
Out[12]: ('foo', [1, 2, 3], True)
```



자료구조와 순차 자료형

Data Structures and Sequences

You can concatenate tuples using the + operator to produce longer tuples:

```
In [13]: (4, None, 'foo') + (6, 0) + ('bar',)
Out[13]: (4, None, 'foo', 6, 0, 'bar')
```

Multiplying a tuple by an integer, as with lists, has the effect of concatenating together that many copies of the tuple:

```
In [14]: ('foo', 'bar') * 4
Out[14]: ('foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'bar')
```

Note that the objects themselves are not copied, only the references to them.



자료구조와 순차 자료형

Data Structures and Sequences

Unpacking tuples

If you try to *assign* to a tuple-like expression of variables, Python will attempt to *unpack* the value on the righthand side of the equals sign:

```
In [15]: tup = (4, 5, 6)
```

```
In [16]: a, b, c = tup
```

```
In [17]: b  
Out[17]: 5
```

Even sequences with nested tuples can be unpacked:

```
In [18]: tup = 4, 5, (6, 7)
```

```
In [19]: a, b, (c, d) = tup
```

```
In [20]: d  
Out[20]: 7
```

Using this functionality you can easily swap variable names, a task which in many languages might look like:

```
tmp = a  
a = b  
b = tmp
```

But, in Python, the swap can be done like this:

```
In [21]: a, b = 1, 2
```

```
In [22]: a  
Out[22]: 1
```

```
In [23]: b  
Out[23]: 2
```

```
In [24]: b, a = a, b
```

```
In [25]: a  
Out[25]: 2
```

```
In [26]: b  
Out[26]: 1
```

A common use of variable unpacking is iterating over sequences of tuples or lists:

```
In [27]: seq = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

```
In [28]: for a, b, c in seq:  
.....:     print('a={0}, b={1}, c={2}'.format(a, b, c))  
a=1, b=2, c=3  
a=4, b=5, c=6  
a=7, b=8, c=9
```



자료구조와 순차 자료형

Data Structures and Sequences

Another common use is returning multiple values from a function. I'll cover this in more detail later.

The Python language recently acquired some more advanced tuple unpacking to help with situations where you may want to “pluck” a few elements from the beginning of a tuple. This uses the special syntax `*rest`, which is also used in function signatures to capture an arbitrarily long list of positional arguments:

```
In [29]: values = 1, 2, 3, 4, 5
```

```
In [30]: a, b, *rest = values
```

```
In [31]: a, b  
Out[31]: (1, 2)
```

```
In [32]: rest  
Out[32]: [3, 4, 5]
```

This `rest` bit is sometimes something you want to discard; there is nothing special about the `rest` name. As a matter of convention, many Python programmers will use the underscore (`_`) for unwanted variables:

```
In [33]: a, b, *_ = values
```

Tuple methods

Since the size and contents of a tuple cannot be modified, it is very light on instance methods. A particularly useful one (also available on lists) is `count`, which counts the number of occurrences of a value:

```
In [34]: a = (1, 2, 2, 2, 3, 4, 2)
```

```
In [35]: a.count(2)  
Out[35]: 4
```



자료구조와 순차 자료형

Data Structures and Sequences

List

In contrast with tuples, lists are variable-length and their contents can be modified in-place. You can define them using square brackets `[]` or using the `list` type function:

```
In [36]: a_list = [2, 3, 7, None]
```

```
In [37]: tup = ('foo', 'bar', 'baz')
```

```
In [38]: b_list = list(tup)
```

```
In [39]: b_list
```

```
Out[39]: ['foo', 'bar', 'baz']
```

```
In [40]: b_list[1] = 'peekaboo'
```

```
In [41]: b_list
```

```
Out[41]: ['foo', 'peekaboo', 'baz']
```

Lists and tuples are semantically similar (though tuples cannot be modified) and can be used interchangeably in many functions.

The `list` function is frequently used in data processing as a way to materialize an iterator or generator expression:

```
In [42]: gen = range(10)
```

```
In [43]: gen
```

```
Out[43]: range(0, 10)
```

```
In [44]: list(gen)
```

```
Out[44]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Concatenating and combining lists

Similar to tuples, adding two lists together with `+` concatenates them:

```
In [57]: [4, None, 'foo'] + [7, 8, (2, 3)]
```

```
Out[57]: [4, None, 'foo', 7, 8, (2, 3)]
```

If you have a list already defined, you can append multiple elements to it using the `extend` method:

```
In [58]: x = [4, None, 'foo']
```

```
In [59]: x.extend([7, 8, (2, 3)])
```

```
In [60]: x
```

```
Out[60]: [4, None, 'foo', 7, 8, (2, 3)]
```

Note that list concatenation by addition is a comparatively expensive operation since a new list must be created and the objects copied over. Using `extend` to append elements to an existing list, especially if you are building up a large list, is usually preferable. Thus,

```
everything = []
for chunk in list_of_lists:
    everything.extend(chunk)
```

is faster than the concatenative alternative:

```
everything = []
for chunk in list_of_lists:
    everything = everything + chunk
```



자료구조와 순차 자료형

Data Structures and Sequences

Adding and removing elements

Elements can be appended to the end of the list with the `append` method:

```
In [45]: b_list.append('dwarf')
```

```
In [46]: b_list
```

```
Out[46]: ['foo', 'peekaboo', 'baz', 'dwarf']
```

Using `insert` you can insert an element at a specific location in the list:

```
In [47]: b_list.insert(1, 'red')
```

```
In [48]: b_list
```

```
Out[48]: ['foo', 'red', 'peekaboo', 'baz', 'dwarf']
```

The insertion index must be between 0 and the length of the list, inclusive.

The inverse operation to `insert` is `pop`, which removes and returns an element at a particular index:

```
In [49]: b_list.pop(2)
```

```
Out[49]: 'peekaboo'
```

```
In [50]: b_list
```

```
Out[50]: ['foo', 'red', 'baz', 'dwarf']
```

Elements can be removed by value with `remove`, which locates the first such value and removes it from the list:

```
In [51]: b_list.append('foo')
```

```
In [52]: b_list
```

```
Out[52]: ['foo', 'red', 'baz', 'dwarf', 'foo']
```

```
In [53]: b_list.remove('foo')
```

```
In [54]: b_list
```

```
Out[54]: ['red', 'baz', 'dwarf', 'foo']
```

If performance is not a concern, by using `append` and `remove`, you can use a Python list as a perfectly suitable “multiset” data structure.

Check if a list contains a value using the `in` keyword:

```
In [55]: 'dwarf' in b_list
```

```
Out[55]: True
```

The keyword `not` can be used to negate `in`:

```
In [56]: 'dwarf' not in b_list
```

```
Out[56]: False
```

Checking whether a list contains a value is a lot slower than doing so with dicts and sets (to be introduced shortly), as Python makes a linear scan across the values of the list, whereas it can check the others (based on hash tables) in constant time.



자료구조와 순차 자료형

Data Structures and Sequences

Sorting

You can sort a list in-place (without creating a new object) by calling its `sort` function:

```
In [61]: a = [7, 2, 5, 1, 3]
```

```
In [62]: a.sort()
```

```
In [63]: a
```

```
Out[63]: [1, 2, 3, 5, 7]
```

`sort` has a few options that will occasionally come in handy. One is the ability to pass a secondary *sort key*—that is, a function that produces a value to use to sort the objects. For example, we could sort a collection of strings by their lengths:

```
In [64]: b = ['saw', 'small', 'He', 'foxes', 'six']
```

```
In [65]: b.sort(key=len)
```

```
In [66]: b
```

```
Out[66]: ['He', 'saw', 'six', 'small', 'foxes']
```

Soon, we'll look at the `sorted` function, which can produce a sorted copy of a general sequence.



자료구조와 순차 자료형

Data Structures and Sequences

Binary search and maintaining a sorted list

The built-in `bisect` module implements binary search and insertion into a sorted list. `bisect.bisect` finds the location where an element should be inserted to keep it sorted, while `bisect.insort` actually inserts the element into that location:

```
In [67]: import bisect
```

```
In [68]: c = [1, 2, 2, 2, 3, 4, 7]
```

```
In [69]: bisect.bisect(c, 2)
```

```
Out[69]: 4
```

```
In [70]: bisect.bisect(c, 5)
```

```
Out[70]: 6
```

```
In [71]: bisect.insort(c, 6)
```

```
In [72]: c
```

```
Out[72]: [1, 2, 2, 2, 3, 4, 6, 7]
```



Slicing

You can select sections of most sequence types by using slice notation, which in its basic form consists of `start:stop` passed to the indexing operator `[]`:

```
In [73]: seq = [7, 2, 3, 7, 5, 6, 0, 1]
```

```
In [74]: seq[1:5]
```

```
Out[74]: [2, 3, 7, 5]
```

Slices can also be assigned to with a sequence:

```
In [75]: seq[3:4] = [6, 3]
```

```
In [76]: seq
```

```
Out[76]: [7, 2, 3, 6, 3, 5, 6, 0, 1]
```

While the element at the `start` index is included, the `stop` index is *not included*, so that the number of elements in the result is `stop - start`.

Either the `start` or `stop` can be omitted, in which case they default to the start of the sequence and the end of the sequence, respectively:

```
In [77]: seq[:5]
```

```
Out[77]: [7, 2, 3, 6, 3]
```

```
In [78]: seq[3:]
```

```
Out[78]: [6, 3, 5, 6, 0, 1]
```

Negative indices slice the sequence relative to the end:

```
In [79]: seq[-4:]
```

```
Out[79]: [5, 6, 0, 1]
```

```
In [80]: seq[-6:-2]
```

```
Out[80]: [6, 3, 5, 6]
```

Slicing semantics takes a bit of getting used to, especially if you're coming from R or MATLAB. See [Figure 3-1](#) for a helpful illustration of slicing with positive and negative integers. In the figure, the indices are shown at the “bin edges” to help show where the slice selections start and stop using positive or negative indices.



자료구조와 순차 자료형

Data Structures and Sequences

A `step` can also be used after a second colon to, say, take every other element:

```
In [81]: seq[::2]  
Out[81]: [7, 3, 3, 6, 1]
```

A clever use of this is to pass `-1`, which has the useful effect of reversing a list or tuple:

```
In [82]: seq[::-1]  
Out[82]: [1, 0, 6, 5, 3, 6, 3, 2, 7]
```

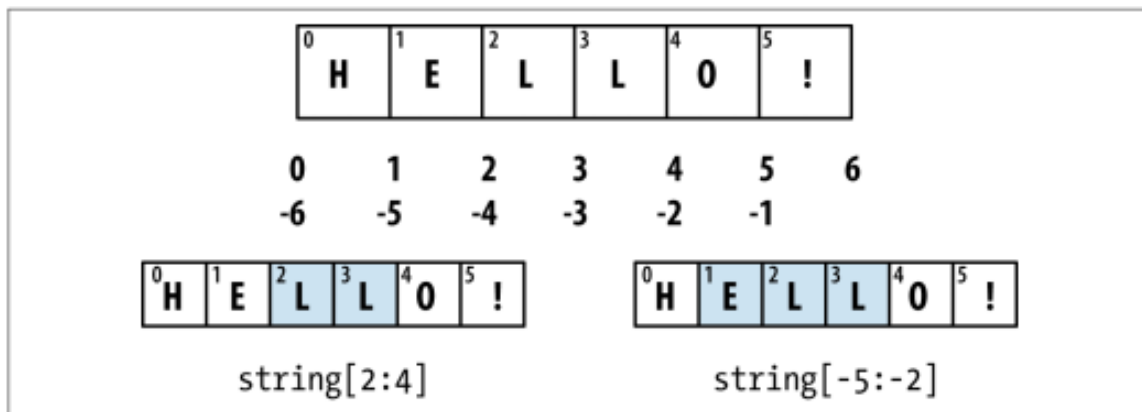


Figure 3-1. Illustration of Python slicing conventions



Built-in Sequence Functions

Python has a handful of useful sequence functions that you should familiarize yourself with and use at any opportunity.

enumerate

It's common when iterating over a sequence to want to keep track of the index of the current item. A do-it-yourself approach would look like:

```
i = 0
for value in collection:
    # do something with value
    i += 1
```

Since this is so common, Python has a built-in function, `enumerate`, which returns a sequence of `(i, value)` tuples:

```
for i, value in enumerate(collection):
    # do something with value
```

When you are indexing data, a helpful pattern that uses `enumerate` is computing a dict mapping the values of a sequence (which are assumed to be unique) to their locations in the sequence:

```
In [83]: some_list = ['foo', 'bar', 'baz']

In [84]: mapping = {}
```

```
In [85]: for i, v in enumerate(some_list):
....:     mapping[v] = i

In [86]: mapping
Out[86]: {'bar': 1, 'baz': 2, 'foo': 0}
```

sorted

The `sorted` function returns a new sorted list from the elements of any sequence:

```
In [87]: sorted([7, 1, 2, 6, 0, 3, 2])
Out[87]: [0, 1, 2, 2, 3, 6, 7]

In [88]: sorted('horse race')
Out[88]: [' ', 'a', 'c', 'e', 'e', 'h', 'o', 'r', 'r', 's']
```

The `sorted` function accepts the same arguments as the `sort` method on lists.



자료구조와 순차 자료형

Data Structures and Sequences

zip

zip “pairs” up the elements of a number of lists, tuples, or other sequences to create a list of tuples:

```
In [89]: seq1 = ['foo', 'bar', 'baz']

In [90]: seq2 = ['one', 'two', 'three']

In [91]: zipped = zip(seq1, seq2)

In [92]: list(zipped)
Out[92]: [('foo', 'one'), ('bar', 'two'), ('baz', 'three')]
```

zip can take an arbitrary number of sequences, and the number of elements it produces is determined by the *shortest* sequence:

```
In [93]: seq3 = [False, True]

In [94]: list(zip(seq1, seq2, seq3))
Out[94]: [('foo', 'one', False), ('bar', 'two', True)]
```

A very common use of zip is simultaneously iterating over multiple sequences, possibly also combined with enumerate:

```
In [95]: for i, (a, b) in enumerate(zip(seq1, seq2)):
.....:     print('{0}: {1}, {2}'.format(i, a, b))
.....:
0: foo, one
1: bar, two
2: baz, three
```

Given a “zipped” sequence, zip can be applied in a clever way to “unzip” the sequence. Another way to think about this is converting a list of *rows* into a list of *columns*. The syntax, which looks a bit magical, is:

```
In [96]: pitchers = [('Nolan', 'Ryan'), ('Roger', 'Clemens'),
.....:                ('Schilling', 'Curt')]

In [97]: first_names, last_names = zip(*pitchers)

In [98]: first_names
Out[98]: ('Nolan', 'Roger', 'Schilling')

In [99]: last_names
Out[99]: ('Ryan', 'Clemens', 'Curt')
```

reversed

reversed iterates over the elements of a sequence in reverse order:

```
In [100]: list(reversed(range(10)))
Out[100]: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Keep in mind that reversed is a generator (to be discussed in some more detail later), so it does not create the reversed sequence until materialized (e.g., with list or a for loop).



자료구조와 순차 자료형

Data Structures and Sequences

dict

`dict` is likely the most important built-in Python data structure. A more common name for it is *hash map* or *associative array*. It is a flexibly sized collection of *key-value* pairs, where *key* and *value* are Python objects. One approach for creating one is to use curly braces `{}` and colons to separate keys and values:

```
In [101]: empty_dict = {}
```

```
In [102]: d1 = {'a': 'some value', 'b': [1, 2, 3, 4]}
```

```
In [103]: d1
```

```
Out[103]: {'a': 'some value', 'b': [1, 2, 3, 4]}
```

You can access, insert, or set elements using the same syntax as for accessing elements of a list or tuple:

```
In [104]: d1[7] = 'an integer'
```

```
In [105]: d1
```

```
Out[105]: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'an integer'}
```

```
In [106]: d1['b']
```

```
Out[106]: [1, 2, 3, 4]
```

You can check if a dict contains a key using the same syntax used for checking whether a list or tuple contains a value:

```
In [107]: 'b' in d1
```

```
Out[107]: True
```

You can delete values either using the `del` keyword or the `pop` method (which simultaneously returns the value and deletes the key):

```
In [108]: d1[5] = 'some value'
```

```
In [109]: d1
```

```
Out[109]: {'a': 'some value',  
           'b': [1, 2, 3, 4],  
           7: 'an integer',  
           5: 'some value'}
```

```
In [110]: d1['dummy'] = 'another value'
```

```
In [111]: d1
```

```
Out[111]: {'a': 'some value',  
           'b': [1, 2, 3, 4],  
           7: 'an integer',  
           5: 'some value',  
           'dummy': 'another value'}
```



자료구조와 순차 자료형

Data Structures and Sequences

```
In [112]: del d1[5]
```

```
In [113]: d1
Out[113]:
{'a': 'some value',
 'b': [1, 2, 3, 4],
 7: 'an integer',
 'dummy': 'another value'}
```

```
In [114]: ret = d1.pop('dummy')
```

```
In [115]: ret
Out[115]: 'another value'
```

```
In [116]: d1
Out[116]: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'an integer'}
```

The keys and values method give you iterators of the dict's keys and values, respectively. While the key-value pairs are not in any particular order, these functions output the keys and values in the same order:

```
In [117]: list(d1.keys())
Out[117]: ['a', 'b', 7]
```

```
In [118]: list(d1.values())
Out[118]: ['some value', [1, 2, 3, 4], 'an integer']
```

You can merge one dict into another using the update method:

```
In [119]: d1.update({'b' : 'foo', 'c' : 12})
```

```
In [120]: d1
Out[120]: {'a': 'some value', 'b': 'foo', 7: 'an integer', 'c': 12}
```

The update method changes dicts in-place, so any existing keys in the data passed to update will have their old values discarded.



자료구조와 순차 자료형

Data Structures and Sequences

Creating dicts from sequences

It's common to occasionally end up with two sequences that you want to pair up element-wise in a dict. As a first cut, you might write code like this:

```
mapping = {}  
for key, value in zip(key_list, value_list):  
    mapping[key] = value
```

Since a dict is essentially a collection of 2-tuples, the dict function accepts a list of 2-tuples:

```
In [121]: mapping = dict(zip(range(5), reversed(range(5))))
```

```
In [122]: mapping
```

```
Out[122]: {0: 4, 1: 3, 2: 2, 3: 1, 4: 0}
```

Later we'll talk about *dict comprehensions*, another elegant way to construct dicts.



자료구조와 순차 자료형

Data Structures and Sequences

Default values

It's very common to have logic like:

```
if key in some_dict:
    value = some_dict[key]
else:
    value = default_value
```

Thus, the dict methods `get` and `pop` can take a default value to be returned, so that the above if-else block can be written simply as:

```
value = some_dict.get(key, default_value)
```

`get` by default will return `None` if the key is not present, while `pop` will raise an exception. With *setting* values, a common case is for the values in a dict to be other collections, like lists. For example, you could imagine categorizing a list of words by their first letters as a dict of lists:

```
In [123]: words = ['apple', 'bat', 'bar', 'atom', 'book']
```

```
In [124]: by_letter = {}
```

```
In [125]: for word in words:
.....:     letter = word[0]
.....:     if letter not in by_letter:
.....:         by_letter[letter] = [word]
.....:     else:
.....:         by_letter[letter].append(word)
```

```
In [126]: by_letter
Out[126]: {'a': ['apple', 'atom'], 'b': ['bat', 'bar', 'book']}
```

The `setdefault` dict method is for precisely this purpose. The preceding for loop can be rewritten as:

```
for word in words:
    letter = word[0]
    by_letter.setdefault(letter, []).append(word)
```

The built-in `collections` module has a useful class, `defaultdict`, which makes this even easier. To create one, you pass a type or function for generating the default value for each slot in the dict:

```
from collections import defaultdict
by_letter = defaultdict(list)
for word in words:
    by_letter[word[0]].append(word)
```




Valid dict key types

While the values of a dict can be any Python object, the keys generally have to be immutable objects like scalar types (int, float, string) or tuples (all the objects in the tuple need to be immutable, too). The technical term here is *hashability*. You can check whether an object is hashable (can be used as a key in a dict) with the hash function:

```
In [127]: hash('string')
Out[127]: 5023931463650008331
```

```
In [128]: hash((1, 2, (2, 3)))
Out[128]: 1097636502276347782
```

```
In [129]: hash((1, 2, [2, 3])) # fails because lists are mutable
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-129-800cd14ba8be> in <module>()
----> 1 hash((1, 2, [2, 3])) # fails because lists are mutable
TypeError: unhashable type: 'list'
```

To use a list as a key, one option is to convert it to a tuple, which can be hashed as long as its elements also can:

```
In [130]: d = {}
```

```
In [131]: d[tuple([1, 2, 3])] = 5
```

```
In [132]: d
Out[132]: {(1, 2, 3): 5}
```



set

A set is an unordered collection of unique elements. You can think of them like dicts, but keys only, no values. A set can be created in two ways: via the `set` function or via a *set literal* with curly braces:

```
In [133]: set([2, 2, 2, 1, 3, 3])
Out[133]: {1, 2, 3}
```

```
In [134]: {2, 2, 2, 1, 3, 3}
Out[134]: {1, 2, 3}
```

Sets support mathematical *set operations* like union, intersection, difference, and symmetric difference. Consider these two example sets:

```
In [135]: a = {1, 2, 3, 4, 5}
```

```
In [136]: b = {3, 4, 5, 6, 7, 8}
```

The union of these two sets is the set of distinct elements occurring in either set. This can be computed with either the `union` method or the `|` binary operator:

```
In [137]: a.union(b)
Out[137]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [138]: a | b
Out[138]: {1, 2, 3, 4, 5, 6, 7, 8}
```

The intersection contains the elements occurring in both sets. The `&` operator or the `intersection` method can be used:

```
In [139]: a.intersection(b)
Out[139]: {3, 4, 5}
```

```
In [140]: a & b
Out[140]: {3, 4, 5}
```

See [Table 3-1](#) for a list of commonly used set methods.



자료구조와 순차 자료형

Data Structures and Sequences

Table 3-1. Python set operations

Function	Alternative syntax	Description
<code>a.add(x)</code>	N/A	Add element <code>x</code> to the set <code>a</code>
<code>a.clear()</code>	N/A	Reset the set <code>a</code> to an empty state, discarding all of its elements
<code>a.remove(x)</code>	N/A	Remove element <code>x</code> from the set <code>a</code>
<code>a.pop()</code>	N/A	Remove an arbitrary element from the set <code>a</code> , raising <code>KeyError</code> if the set is empty
<code>a.union(b)</code>	<code>a b</code>	All of the unique elements in <code>a</code> and <code>b</code>
<code>a.update(b)</code>	<code>a = b</code>	Set the contents of <code>a</code> to be the union of the elements in <code>a</code> and <code>b</code>
<code>a.intersection(b)</code>	<code>a & b</code>	All of the elements in <i>both</i> <code>a</code> and <code>b</code>
<code>a.intersection_update(b)</code>	<code>a &= b</code>	Set the contents of <code>a</code> to be the intersection of the elements in <code>a</code> and <code>b</code>
<code>a.difference(b)</code>	<code>a - b</code>	The elements in <code>a</code> that are not in <code>b</code>
<code>a.difference_update(b)</code>	<code>a -= b</code>	Set <code>a</code> to the elements in <code>a</code> that are not in <code>b</code>
<code>a.symmetric_difference(b)</code>	<code>a ^ b</code>	All of the elements in either <code>a</code> or <code>b</code> but <i>not both</i>
<code>a.symmetric_difference_update(b)</code>	<code>a ^= b</code>	Set <code>a</code> to contain the elements in either <code>a</code> or <code>b</code> but <i>not both</i>
<code>a.issubset(b)</code>	N/A	True if the elements of <code>a</code> are all contained in <code>b</code>
<code>a.issuperset(b)</code>	N/A	True if the elements of <code>b</code> are all contained in <code>a</code>
<code>a.isdisjoint(b)</code>	N/A	True if <code>a</code> and <code>b</code> have no elements in common



자료구조와 순차 자료형

Data Structures and Sequences

All of the logical set operations have in-place counterparts, which enable you to replace the contents of the set on the left side of the operation with the result. For very large sets, this may be more efficient:

```
In [141]: c = a.copy()

In [142]: c |= b

In [143]: c
Out[143]: {1, 2, 3, 4, 5, 6, 7, 8}

In [144]: d = a.copy()

In [145]: d &= b

In [146]: d
Out[146]: {3, 4, 5}
```

Like dicts, set elements generally must be immutable. To have list-like elements, you must convert it to a tuple:

```
In [147]: my_data = [1, 2, 3, 4]

In [148]: my_set = {tuple(my_data)}

In [149]: my_set
Out[149]: {(1, 2, 3, 4)}
```

You can also check if a set is a subset of (is contained in) or a superset of (contains all elements of) another set:

```
In [150]: a_set = {1, 2, 3, 4, 5}

In [151]: {1, 2, 3}.issubset(a_set)
Out[151]: True

In [152]: a_set.issuperset({1, 2, 3})
Out[152]: True
```

Sets are equal if and only if their contents are equal:

```
In [153]: {1, 2, 3} == {3, 2, 1}
Out[153]: True
```



List, Set, and Dict Comprehensions

List comprehensions are one of the most-loved Python language features. They allow you to concisely form a new list by filtering the elements of a collection, transforming the elements passing the filter in one concise expression. They take the basic form:

```
[expr for val in collection if condition]
```

This is equivalent to the following for loop:

```
result = []
for val in collection:
    if condition:
        result.append(expr)
```

The filter condition can be omitted, leaving only the expression. For example, given a list of strings, we could filter out strings with length 2 or less and also convert them to uppercase like this:

```
In [154]: strings = ['a', 'as', 'bat', 'car', 'dove', 'python']
```

```
In [155]: [x.upper() for x in strings if len(x) > 2]
```

```
Out[155]: ['BAT', 'CAR', 'DOVE', 'PYTHON']
```

Set and dict comprehensions are a natural extension, producing sets and dicts in an idiomatically similar way instead of lists. A dict comprehension looks like this:

```
dict_comp = {key-expr : value-expr for value in collection
              if condition}
```

A set comprehension looks like the equivalent list comprehension except with curly braces instead of square brackets:

```
set_comp = {expr for value in collection if condition}
```

Like list comprehensions, set and dict comprehensions are mostly conveniences, but they similarly can make code both easier to write and read. Consider the list of strings from before. Suppose we wanted a set containing just the lengths of the strings contained in the collection; we could easily compute this using a set comprehension:

```
In [156]: unique_lengths = {len(x) for x in strings}
```

```
In [157]: unique_lengths
Out[157]: {1, 2, 3, 4, 6}
```

We could also express this more functionally using the `map` function, introduced shortly:

```
In [158]: set(map(len, strings))
Out[158]: {1, 2, 3, 4, 6}
```

As a simple dict comprehension example, we could create a lookup map of these strings to their locations in the list:

```
In [159]: loc_mapping = {val : index for index, val in enumerate(strings)}
```

```
In [160]: loc_mapping
Out[160]: {'a': 0, 'as': 1, 'bat': 2, 'car': 3, 'dove': 4, 'python': 5}
```



자료구조와 순차 자료형

Data Structures and Sequences

Nested list comprehensions

Suppose we have a list of lists containing some English and Spanish names:

```
In [161]: all_data = [['John', 'Emily', 'Michael', 'Mary', 'Steven'],  
.....:               ['Maria', 'Juan', 'Javier', 'Natalia', 'Pilar']]
```

You might have gotten these names from a couple of files and decided to organize them by language. Now, suppose we wanted to get a single list containing all names with two or more e's in them. We could certainly do this with a simple for loop:

```
names_of_interest = []  
for names in all_data:  
    enough_es = [name for name in names if name.count('e') >= 2]  
    names_of_interest.extend(enough_es)
```

You can actually wrap this whole operation up in a single *nested list comprehension*, which will look like:

```
In [162]: result = [name for names in all_data for name in names  
.....:               if name.count('e') >= 2]  
  
In [163]: result  
Out[163]: ['Steven']
```

At first, nested list comprehensions are a bit hard to wrap your head around. The for parts of the list comprehension are arranged according to the order of nesting, and any filter condition is put at the end as before. Here is another example where we “flatten” a list of tuples of integers into a simple list of integers:

```
In [164]: some_tuples = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]  
  
In [165]: flattened = [x for tup in some_tuples for x in tup]  
  
In [166]: flattened  
Out[166]: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Keep in mind that the order of the for expressions would be the same if you wrote a nested for loop instead of a list comprehension:

```
flattened = []  
  
for tup in some_tuples:  
    for x in tup:  
        flattened.append(x)
```

You can have arbitrarily many levels of nesting, though if you have more than two or three levels of nesting you should probably start to question whether this makes sense from a code readability standpoint. It's important to distinguish the syntax just shown from a list comprehension inside a list comprehension, which is also perfectly valid:

```
In [167]: [[x for x in tup] for tup in some_tuples]  
Out[167]: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

This produces a list of lists, rather than a flattened list of all of the inner elements.

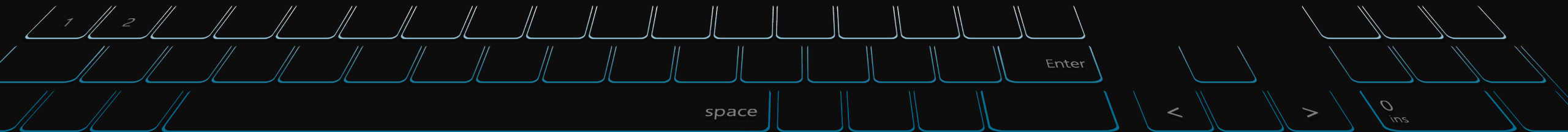
Q&A and Break Time

질의응답 및 휴식 시간 (5분)



2부 : 함수, 파일, 운영체제

- 1) 함수
- 2) 파일
- 3) 운영체제





Functions are the primary and most important method of code organization and reuse in Python. As a rule of thumb, if you anticipate needing to repeat the same or very similar code more than once, it may be worth writing a reusable function. Functions can also help make your code more readable by giving a name to a group of Python statements.

Functions are declared with the `def` keyword and returned from with the `return` keyword:

```
def my_function(x, y, z=1.5):  
    if z > 1:  
        return z * (x + y)  
    else:  
        return z / (x + y)
```

There is no issue with having multiple `return` statements. If Python reaches the end of a function without encountering a `return` statement, `None` is returned automatically.

Each function can have *positional* arguments and *keyword* arguments. Keyword arguments are most commonly used to specify default values or optional arguments. In the preceding function, `x` and `y` are positional arguments while `z` is a keyword argument. This means that the function can be called in any of these ways:

```
my_function(5, 6, z=0.7)  
my_function(3.14, 7, 3.5)  
my_function(10, 20)
```

The main restriction on function arguments is that the keyword arguments *must* follow the positional arguments (if any). You can specify keyword arguments in any order; this frees you from having to remember which order the function arguments were specified in and only what their names are.



Namespaces, Scope, and Local Functions

Functions can access variables in two different scopes: *global* and *local*. An alternative and more descriptive name describing a variable scope in Python is a *namespace*. Any variables that are assigned within a function by default are assigned to the local namespace. The local namespace is created when the function is called and immediately populated by the function's arguments. After the function is finished, the local namespace is destroyed (with some exceptions that are outside the purview of this chapter). Consider the following function:

```
def func():  
    a = []  
    for i in range(5):  
        a.append(i)
```

When `func()` is called, the empty list `a` is created, five elements are appended, and then `a` is destroyed when the function exits. Suppose instead we had declared `a` as follows:

```
a = []  
def func():  
    for i in range(5):  
        a.append(i)
```

Assigning variables outside of the function's scope is possible, but those variables must be declared as global via the `global` keyword:

```
In [168]: a = None
```

```
In [169]: def bind_a_variable():  
.....:     global a  
.....:     a = []  
.....:     bind_a_variable()  
.....:
```

```
In [170]: print(a)  
[]
```



Returning Multiple Values

When I first programmed in Python after having programmed in Java and C++, one of my favorite features was the ability to return multiple values from a function with simple syntax. Here's an example:

```
def f():  
    a = 5  
    b = 6  
    c = 7  
    return a, b, c
```

```
a, b, c = f()
```

In data analysis and other scientific applications, you may find yourself doing this often. What's happening here is that the function is actually just returning *one* object, namely a tuple, which is then being unpacked into the result variables. In the preceding example, we could have done this instead:

```
return_value = f()
```

In this case, `return_value` would be a 3-tuple with the three returned variables. A potentially attractive alternative to returning multiple values like before might be to return a dict instead:

```
def f():  
    a = 5  
    b = 6  
    c = 7  
    return {'a' : a, 'b' : b, 'c' : c}
```

This alternative technique can be useful depending on what you are trying to do.



Functions Are Objects

Since Python functions are objects, many constructs can be easily expressed that are difficult to do in other languages. Suppose we were doing some data cleaning and needed to apply a bunch of transformations to the following list of strings:

```
In [171]: states = ['  Alabama ', 'Georgia!', 'Georgia', 'georgia', 'FlOrIda',
.....:             'south carolina##', 'West virginia?']
```

Anyone who has ever worked with user-submitted survey data has seen messy results like these. Lots of things need to happen to make this list of strings uniform and ready for analysis: stripping whitespace, removing punctuation symbols, and standardizing on proper capitalization. One way to do this is to use built-in string methods along with the `re` standard library module for regular expressions:

```
import re

def clean_strings(strings):
    result = []
    for value in strings:
        value = value.strip()
        value = re.sub('[!#?]', '', value)
        value = value.title()
        result.append(value)
    return result
```

The result looks like this:

```
In [173]: clean_strings(states)
Out[173]:
['Alabama',
'Georgia',
'Georgia',
'Georgia',
'Georgia',
'Florida',
'South Carolina',
'West Virginia']
```



An alternative approach that you may find useful is to make a list of the operations you want to apply to a particular set of strings:

```
def remove_punctuation(value):  
    return re.sub('[!#?]', '', value)  
  
clean_ops = [str.strip, remove_punctuation, str.title]  
  
def clean_strings(strings, ops):  
    result = []  
    for value in strings:  
        for function in ops:  
            value = function(value)  
        result.append(value)  
    return result
```

Then we have the following:

```
In [175]: clean_strings(states, clean_ops)  
Out[175]:  
['Alabama',  
 'Georgia',  
 'Georgia',  
 'Georgia',  
 'Florida',  
 'South Carolina',  
 'West Virginia']
```

A more *functional* pattern like this enables you to easily modify how the strings are transformed at a very high level. The `clean_strings` function is also now more reusable and generic.

You can use functions as arguments to other functions like the built-in `map` function, which applies a function to a sequence of some kind:

```
In [176]: for x in map(remove_punctuation, states):  
    .....:     print(x)  
Alabama  
Georgia  
Georgia  
georgia  
FlOrIda  
south carolina  
West virginia
```



Anonymous (Lambda) Functions

Python has support for so-called *anonymous* or *lambda* functions, which are a way of writing functions consisting of a single statement, the result of which is the return value. They are defined with the `lambda` keyword, which has no meaning other than “we are declaring an anonymous function”:

```
def short_function(x):  
    return x * 2
```

```
equiv_anon = lambda x: x * 2
```

I usually refer to these as lambda functions in the rest of the book. They are especially convenient in data analysis because, as you’ll see, there are many cases where data transformation functions will take functions as arguments. It’s often less typing (and clearer) to pass a lambda function as opposed to writing a full-out function declaration or even assigning the lambda function to a local variable. For example, consider this silly example:

```
def apply_to_list(some_list, f):  
    return [f(x) for x in some_list]
```

```
ints = [4, 0, 1, 5, 6]  
apply_to_list(ints, lambda x: x * 2)
```

You could also have written `[x * 2 for x in ints]`, but here we were able to succinctly pass a custom operator to the `apply_to_list` function.

As another example, suppose you wanted to sort a collection of strings by the number of distinct letters in each string:

```
In [177]: strings = ['foo', 'card', 'bar', 'aaaa', 'abab']
```

Here we could pass a lambda function to the list’s `sort` method:

```
In [178]: strings.sort(key=lambda x: len(set(list(x))))
```

```
In [179]: strings
```

```
Out[179]: ['aaaa', 'foo', 'abab', 'bar', 'card']
```



Currying: Partial Argument Application

Currying is computer science jargon (named after the mathematician Haskell Curry) that means deriving new functions from existing ones by *partial argument application*. For example, suppose we had a trivial function that adds two numbers together:

```
def add_numbers(x, y):  
    return x + y
```

Using this function, we could derive a new function of one variable, `add_five`, that adds 5 to its argument:

```
add_five = lambda y: add_numbers(5, y)
```

The second argument to `add_numbers` is said to be *curried*. There's nothing very fancy here, as all we've really done is define a new function that calls an existing function. The built-in `functools` module can simplify this process using the `partial` function:

```
from functools import partial  
add_five = partial(add_numbers, 5)
```




Generators

Having a consistent way to iterate over sequences, like objects in a list or lines in a file, is an important Python feature. This is accomplished by means of the *iterator protocol*, a generic way to make objects iterable. For example, iterating over a dict yields the dict keys:

```
In [180]: some_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
In [181]: for key in some_dict:
.....:     print(key)
a
b
c
```

When you write `for key in some_dict`, the Python interpreter first attempts to create an iterator out of `some_dict`:

```
In [182]: dict_iterator = iter(some_dict)

In [183]: dict_iterator
Out[183]: <dict_keyiterator at 0x7fbbd5a9f908>
```

An iterator is any object that will yield objects to the Python interpreter when used in a context like a `for` loop. Most methods expecting a list or list-like object will also accept any iterable object. This includes built-in methods such as `min`, `max`, and `sum`, and type constructors like `list` and `tuple`:

```
In [184]: list(dict_iterator)
Out[184]: ['a', 'b', 'c']
```

A *generator* is a concise way to construct a new iterable object. Whereas normal functions execute and return a single result at a time, generators return a sequence of multiple results lazily, pausing after each one until the next one is requested. To create a generator, use the `yield` keyword instead of `return` in a function:

```
def squares(n=10):
    print('Generating squares from 1 to {}'.format(n ** 2))
    for i in range(1, n + 1):
        yield i ** 2
```

When you actually call the generator, no code is immediately executed:

```
In [186]: gen = squares()

In [187]: gen
Out[187]: <generator object squares at 0x7fbbd5ab4570>
```

It is not until you request elements from the generator that it begins executing its code:

```
In [188]: for x in gen:
.....:     print(x, end=' ')
Generating squares from 1 to 100
1 4 9 16 25 36 49 64 81 100
```




Generator expressions

Another even more concise way to make a generator is by using a *generator expression*. This is a generator analogue to list, dict, and set comprehensions; to create one, enclose what would otherwise be a list comprehension within parentheses instead of brackets:

```
In [189]: gen = (x ** 2 for x in range(100))
```

```
In [190]: gen
```

```
Out[190]: <generator object <genexpr> at 0x7fbbd5ab29e8>
```

This is completely equivalent to the following more verbose generator:

```
def _make_gen():  
    for x in range(100):  
        yield x ** 2  
gen = _make_gen()
```

Generator expressions can be used instead of list comprehensions as function arguments in many cases:

```
In [191]: sum(x ** 2 for x in range(100))
```

```
Out[191]: 328350
```

```
In [192]: dict((i, i ** 2) for i in range(5))
```

```
Out[192]: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```



itertools module

The standard library `itertools` module has a collection of generators for many common data algorithms. For example, `groupby` takes any sequence and a function, grouping consecutive elements in the sequence by return value of the function. Here's an example:

```
In [193]: import itertools

In [194]: first_letter = lambda x: x[0]

In [195]: names = ['Alan', 'Adam', 'Wes', 'Will', 'Albert', 'Steven']

In [196]: for letter, names in itertools.groupby(names, first_letter):
.....:     print(letter, list(names)) # names is a generator
A ['Alan', 'Adam']
W ['Wes', 'Will']
A ['Albert']
S ['Steven']
```

See [Table 3-2](#) for a list of a few other `itertools` functions I've frequently found helpful. You may like to check out [the official Python documentation](#) for more on this useful built-in utility module.

Table 3-2. Some useful `itertools` functions

Function	Description
<code>combinations(iterable, k)</code>	Generates a sequence of all possible k-tuples of elements in the iterable, ignoring order and without replacement (see also the companion function <code>combinations_with_replacement</code>)
<code>permutations(iterable, k)</code>	Generates a sequence of all possible k-tuples of elements in the iterable, respecting order
<code>groupby(iterable[, keyfunc])</code>	Generates (key, sub-iterator) for each unique key
<code>product(*iterables, repeat=1)</code>	Generates the Cartesian product of the input iterables as tuples, similar to a nested for loop



Errors and Exception Handling

Handling Python errors or *exceptions* gracefully is an important part of building robust programs. In data analysis applications, many functions only work on certain kinds of input. As an example, Python's `float` function is capable of casting a string to a floating-point number, but fails with `ValueError` on improper inputs:

```
In [197]: float('1.2345')
Out[197]: 1.2345

In [198]: float('something')
-----
ValueError                                Traceback (most recent call last)
<ipython-input-198-439904410854> in <module>()
----> 1 float('something')
ValueError: could not convert string to float: 'something'
```

Suppose we wanted a version of `float` that fails gracefully, returning the input argument. We can do this by writing a function that encloses the call to `float` in a `try/except` block:

```
def attempt_float(x):
    try:
        return float(x)
    except:
        return x
```

The code in the `except` part of the block will only be executed if `float(x)` raises an exception:

```
In [200]: attempt_float('1.2345')
Out[200]: 1.2345
```

```
In [201]: attempt_float('something')
Out[201]: 'something'
```

You might notice that `float` can raise exceptions other than `ValueError`:

```
In [202]: float((1, 2))
-----
TypeError                                Traceback (most recent call last)
<ipython-input-202-842079ebb635> in <module>()
----> 1 float((1, 2))
TypeError: float() argument must be a string or a number, not 'tuple'
```

You might want to only suppress `ValueError`, since a `TypeError` (the input was not a string or numeric value) might indicate a legitimate bug in your program. To do that, write the exception type after `except`:

```
def attempt_float(x):
    try:
        return float(x)
    except ValueError:
        return x
```

We have then:

```
In [204]: attempt_float((1, 2))
-----
TypeError                                Traceback (most recent call last)
<ipython-input-204-9bdfd730cead> in <module>()
----> 1 attempt_float((1, 2))
<ipython-input-203-3e06b8379b6b> in attempt_float(x)
      1 def attempt_float(x):
      2     try:
----> 3         return float(x)
      4     except ValueError:
      5         return x
TypeError: float() argument must be a string or a number, not 'tuple'
```



You can catch multiple exception types by writing a tuple of exception types instead (the parentheses are required):

```
def attempt_float(x):  
    try:  
        return float(x)  
    except (TypeError, ValueError):  
        return x
```

In some cases, you may not want to suppress an exception, but you want some code to be executed regardless of whether the code in the try block succeeds or not. To do this, use finally:

```
f = open(path, 'w')  
  
try:  
    write_to_file(f)  
finally:  
    f.close()
```

Here, the file handle `f` will *always* get closed. Similarly, you can have code that executes only if the try: block succeeds using else:

```
f = open(path, 'w')  
  
try:  
    write_to_file(f)  
except:  
    print('Failed')  
else:  
    print('Succeeded')  
finally:  
    f.close()
```



파일 & 운영체제

Files and OS(Operation System)

Most of this book uses high-level tools like `pandas.read_csv` to read data files from disk into Python data structures. However, it's important to understand the basics of how to work with files in Python. Fortunately, it's very simple, which is one reason why Python is so popular for text and file munging.

To open a file for reading or writing, use the built-in `open` function with either a relative or absolute file path:

```
In [207]: path = 'examples/segismundo.txt'
```

```
In [208]: f = open(path)
```

By default, the file is opened in read-only mode `'r'`. We can then treat the file handle `f` like a list and iterate over the lines like so:

```
for line in f:
    pass
```

The lines come out of the file with the end-of-line (EOL) markers intact, so you'll often see code to get an EOL-free list of lines in a file like:

```
In [209]: lines = [x.rstrip() for x in open(path)]
```

```
In [210]: lines
```

```
Out[210]:
```

```
['Sueña el rico en su riqueza,',
 'que más cuidados le ofrece;',
 '',
 'sueña el pobre que padece',
 'su miseria y su pobreza;',
 '',
 'sueña el que a medrar empieza,',
 'sueña el que afana y pretende,',
 'sueña el que agravia y ofende,',
 '',
 'y en el mundo, en conclusión,',
 'todos sueñan lo que son,',
 'aunque ninguno lo entiende.',
 '']
```



When you use `open` to create file objects, it is important to explicitly close the file when you are finished with it. Closing the file releases its resources back to the operating system:

```
In [211]: f.close()
```

One of the ways to make it easier to clean up open files is to use the `with` statement:

```
In [212]: with open(path) as f:
.....:     lines = [x.rstrip() for x in f]
```

This will automatically close the file `f` when exiting the `with` block.

If we had typed `f = open(path, 'w')`, a *new file* at *examples/segismundo.txt* would have been created (be careful!), overwriting any one in its place. There is also the `'x'` file mode, which creates a writable file but fails if the file path already exists. See [Table 3-3](#) for a list of all valid file read/write modes.

For readable files, some of the most commonly used methods are `read`, `seek`, and `tell`. `read` returns a certain number of characters from the file. What constitutes a “character” is determined by the file’s encoding (e.g., UTF-8) or simply raw bytes if the file is opened in binary mode:

```
In [213]: f = open(path)
```

```
In [214]: f.read(10)
Out[214]: 'Sueña el r'
```

```
In [215]: f2 = open(path, 'rb') # Binary mode
```

```
In [216]: f2.read(10)
Out[216]: b'Sue\xc3\xb1a el '
```

The `read` method advances the file handle’s position by the number of bytes read. `tell` gives you the current position:

```
In [217]: f.tell()
Out[217]: 11
```

```
In [218]: f2.tell()
Out[218]: 10
```




Even though we read 10 characters from the file, the position is 11 because it took that many bytes to decode 10 characters using the default encoding. You can check the default encoding in the sys module:

```
In [219]: import sys
```

```
In [220]: sys.getdefaultencoding()
```

```
Out[220]: 'utf-8'
```

seek changes the file position to the indicated byte in the file:

```
In [221]: f.seek(3)
```

```
Out[221]: 3
```

```
In [222]: f.read(1)
```

```
Out[222]: 'ñ'
```

Lastly, we remember to close the files:

```
In [223]: f.close()
```

```
In [224]: f2.close()
```

Table 3-3. Python file modes

Mode	Description
r	Read-only mode
w	Write-only mode; creates a new file (erasing the data for any file with the same name)
x	Write-only mode; creates a new file, but fails if the file path already exists
a	Append to existing file (create the file if it does not already exist)
r+	Read and write
b	Add to mode for binary files (i.e., 'rb' or 'wb')
t	Text mode for files (automatically decoding bytes to Unicode). This is the default if not specified. Add t to other modes to use this (i.e., 'rt' or 'xt')



파일 & 운영체제

Files and OS(Operation System)

To write text to a file, you can use the file's `write` or `writelines` methods. For example, we could create a version of `prof_mod.py` with no blank lines like so:

```
In [225]: with open('tmp.txt', 'w') as handle:
.....:     handle.writelines(x for x in open(path) if len(x) > 1)
```

```
In [226]: with open('tmp.txt') as f:
.....:     lines = f.readlines()
```

```
In [227]: lines
```

```
Out[227]:
```

```
['Sueña el rico en su riqueza,\n',
'que más cuidados le ofrece;\n',
'sueña el pobre que padece\n',
'su miseria y su pobreza;\n',
'sueña el que a medrar empieza,\n',
'sueña el que afana y pretende,\n',
'sueña el que agravia y ofende,\n',
'y en el mundo, en conclusión,\n',
'todos sueñan lo que son,\n',
'aunque ninguno lo entiende.\n']
```

See [Table 3-4](#) for many of the most commonly used file methods.

Table 3-4. Important Python file methods or attributes

Method	Description
<code>read([size])</code>	Return data from file as a string, with optional <code>size</code> argument indicating the number of bytes to read
<code>readlines([size])</code>	Return list of lines in the file, with optional <code>size</code> argument
<code>write(str)</code>	Write passed string to file
<code>writelines(strings)</code>	Write passed sequence of strings to the file
<code>close()</code>	Close the handle
<code>flush()</code>	Flush the internal I/O buffer to disk
<code>seek(pos)</code>	Move to indicated file position (integer)
<code>tell()</code>	Return current file position as integer
<code>closed</code>	True if the file is closed



Bytes and Unicode with Files

The default behavior for Python files (whether readable or writable) is *text mode*, which means that you intend to work with Python strings (i.e., Unicode). This contrasts with *binary mode*, which you can obtain by appending `b` onto the file mode. Let's look at the file (which contains non-ASCII characters with UTF-8 encoding) from the previous section:

```
In [230]: with open(path) as f:
.....:     chars = f.read(10)
```

```
In [231]: chars
Out[231]: 'Sueña el r'
```

UTF-8 is a variable-length Unicode encoding, so when I requested some number of characters from the file, Python reads enough bytes (which could be as few as 10 or as many as 40 bytes) from the file to decode that many characters. If I open the file in `'rb'` mode instead, `read` requests exact numbers of bytes:

```
In [232]: with open(path, 'rb') as f:
.....:     data = f.read(10)
```

```
In [233]: data
Out[233]: b'Sue\xc3\xb1a el '
```

Depending on the text encoding, you may be able to decode the bytes to a `str` object yourself, but only if each of the encoded Unicode characters is fully formed:

```
In [234]: data.decode('utf8')
Out[234]: 'Sueña el '
```

```
In [235]: data[:4].decode('utf8')
```

```
-----
UnicodeDecodeError                                Traceback (most recent call last)
<ipython-input-235-300e0af10bb7> in <module>()
----> 1 data[:4].decode('utf8')
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc3 in position 3: unexpected end of data
```



파일 & 운영체제

Files and OS(Operation System)

Text mode, combined with the encoding option of open, provides a convenient way to convert from one Unicode encoding to another:

```
In [236]: sink_path = 'sink.txt'

In [237]: with open(path) as source:
.....:     with open(sink_path, 'xt', encoding='iso-8859-1') as sink:
.....:         sink.write(source.read())

In [238]: with open(sink_path, encoding='iso-8859-1') as f:
.....:     print(f.read(10))
Sueña el r
```

Beware using seek when opening files in any mode other than binary. If the file position falls in the middle of the bytes defining a Unicode character, then subsequent reads will result in an error:

```
In [240]: f = open(path)

In [241]: f.read(5)
Out[241]: 'Sueña'

In [242]: f.seek(4)
Out[242]: 4

In [243]: f.read(1)
-----
UnicodeDecodeError                                Traceback (most recent call last)
<ipython-input-243-7841103e33f5> in <module>()
----> 1 f.read(1)
/miniconda/envs/book-env/lib/python3.6/codecs.py in decode(self, input, final)
    319         # decode input (taking the buffer into account)
    320         data = self.buffer + input
--> 321         (result, consumed) = self._buffer_decode(data, self.errors, final)
    322
    323         # keep undecoded input until the next call
    324         self.buffer = data[consumed:]
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb1 in position 0: invalid s
tart byte

In [244]: f.close()
```

If you find yourself regularly doing data analysis on non-ASCII text data, mastering Python's Unicode functionality will prove valuable. See [Python's online documentation](#) for much more.

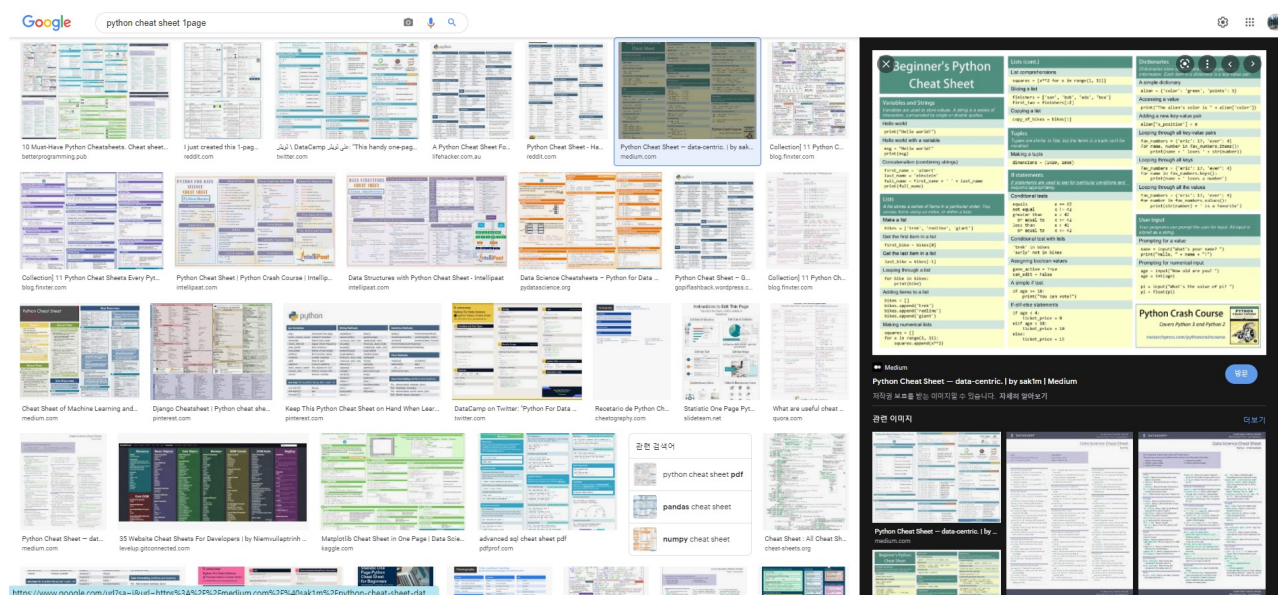


추천 사이트 공지

Homepage w/ Recommendation

[\[LINK\] 실용 파이썬 프로그래밍: 프로그래밍 유경험자를 위한 강좌](#)

[\[LINK\] 10 Must-Have Python Cheatsheets](#)



Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

